

Notification Service

Рефакторинг in-memory очереди отложенных уведомлений

Автор

Группа 20 (с М3102)

Ручкин Олег М3102
Ланских Глеб М3102
Арвачев Михаил М3102
Шебалина Алиса М3102

Что было сделано

ОБЯЗАТЕЛЬНЫЕ ЗАДАЧИ

- ✓ **1. Worker таймаутит на /v1/duе**
замена полного scan + sort на индекс готовности
- ✓ **2. Неверный порядок due**
send_at asc, priority desc, created_at asc, id asc
- ✓ **3. Повторный POST создаёт дубль**
идемпотентность активного id
- ✓ **4. Cancelled тормозят hot path**
физическое удаление из due_index_
- ✓ **5. После sent/cancel нельзя пересоздать**
terminal state не блокирует новый POST
- ✓ **6. Все ручки сериализованы mutex**
shared_mutex: read/write locks
- ✓ **7. No-ready запрос дорогой**
проверка первого элемента индекса

Публичное API сохранено

БОНУСНЫЕ ЗАДАНИЯ НЕ ВЫПОЛНЯЛИСЬ

РАСПРЕДЕЛЕНИЕ ЗАДАЧ

Олег: задачи 1 and 2

hot path /due и строгий порядок

Глеб: задачи 3 and 4

идемпотентность POST и cancel

Миша: задачи 5 and 6

terminal states и shared_mutex

Алиса: задача 7 + проверка

no-ready, тесты, load test

Что улучшилось: метрики + обозначения асимптотик

LOAD RPS

1334 → 7686

5.76× выше

DUE P99

158.59 → 27.71 ms

5.72× быстрее

DUE P95

117.04 → 17.5 ms

6.69× быстрее

ERROR RATE

39.07% → 6.69%

5.84× ниже

TESTS

13 / 13

DISABLED включены

НАГРУЗКА

Duration почти одинаковая: 70.8s baseline и 70.5s после правок

Изначально: Total 94 427, Errors 36 896, RPS 1334.6

После: Total 541 788, Errors 36 245, RPS 7686.1

Successful RPS: 812.6 → 7170.8, то есть 8.82× выше.

ГЛАВНАЯ АСИМПТОТИКА

Было

 $O(N + D \log D)$ 

Стало

 $O(K)$, no-ready $O(1)$

ОБОЗНАЧЕНИЯ АСИМПТОТИК

N — вся история id в schedule_

A — активные pending в due_index_

D — due-кандидаты с send_at ≤ now

K — limit, размер пачки worker-a



Задача 1: worker таймаутил на /v1/due

SRE #1

README: worker просит limit=100, но baseline проходит по всей истории и сортирует всех кандидатов. Маленький limit никак не ограничивает работу hot path.

Было

 $O(N + D \log D)$ 

Стало

 $O(K)$ после построения
индекса

Цена перенесена на изменение очереди: add/cancel/sent поддерживают std::set за $O(\log A)$, зато polling due перестал зависеть от N.

БЫЛО: scan schedule_ + sort всех candidates

```
std::vector<DueNotification> result;
result.reserve(schedule_.size());
for (const auto& id : schedule_) {
    auto it = notifications_.find(id);
    if (it == notifications_.end()) continue;
    const auto& n = it->second;
    if (n.send_at > now) continue;
    if (cancelled_.count(id) != 0 || sent_.count(id) != 0) continue;
    result.push_back(toDue(n));
}
std::sort(result.begin(), result.end(), comparator);
if (result.size() > limit) result.resize(limit);
```

СТАЛО: идём только по началу due_index_

```
result.reserve(std::min(limit, due_index_.size()));
for (auto it = due_index_.begin();
     it != due_index_.end() && result.size() < limit; ++it) {
    if (it->send_at > now) {
        break;
    }

    auto notification_it = notifications_.find(it->id);
    if (notification_it == notifications_.end()) {
        continue;
    }

    const auto& notification =
        notification_it->second.notification;
    if (notification.status != NotificationStatus::Pending) {
        continue;
    }
    result.push_back(toDue(notification));
}
return result;
```

Задача 2: строгий порядок выдачи due

Backend #2

README: send_at asc → priority desc → created_at asc → id asc. В baseline сортировались только send_at и id, поэтому при одинаковом send_at приоритеты и created_at игнорировались.

Было

`sort: send_at, id`

Стало

`set key: send_at, priority desc, created_at, id`

БЫЛО: неполный comparator

```
std::sort(result.begin(), result.end(),
          [](const DueNotification& a, const DueNotification& b) {
            if (a.send_at != b.send_at) return a.send_at < b.send_at;
            return a.id < b.id;
          });
```

СТАЛО: порядок зафиксирован в индексе

```
bool NotificationService::DueIndexEntryLess::operator()(
    const DueIndexEntry& lhs,
    const DueIndexEntry& rhs) const {
    if (lhs.send_at != rhs.send_at) {
        return lhs.send_at < rhs.send_at;
    }
    if (lhs.priority != rhs.priority) {
        return lhs.priority > rhs.priority;
    }
    if (lhs.created_at != rhs.created_at) {
        return lhs.created_at < rhs.created_at;
    }
    return lhs.id < rhs.id;
}
```

Асимптотика не ухудшает due: сортировки результата больше нет. Порядок обеспечивается при insert в std::set за $O(\log A)$, а чтение первых K элементов уже отсортировано.

Задача 3: повторный POST с тем же id не создаёт дубль

QA #3

README: клиент отправил POST, получил сетевой таймаут и ретраит тот же id. Сервис должен вернуть состояние “как будто POST был один раз”: не перетирать активную задачу и не добавлять второй id в hot path.

Было

`duplicate due cost: O(retries)`

Стало

`duplicate POST: O(1) avg, due без дублей`

БЫЛО: каждый POST пишет id в schedule_

```
void NotificationService::add(Notification notification) {
    std::lock_guard<std::mutex> lk(mu_);
    notification.status = NotificationStatus::Pending;
    schedule_.push_back(notification.id);
    notifications_[notification.id] = std::move(notification);
}
```

Ключевой инвариант: для pending id существует максимум один элемент в due_index_.

СТАЛО: активный id идемпотентен

```
void NotificationService::add(Notification notification) {
    std::unique_lock<std::shared_mutex> lock(mu_);

    const std::string key = notification.id;
    auto it = notifications_.find(key);
    if (it != notifications_.end() &&
        it->second.notification.status ==
            NotificationStatus::Pending) {
        return;
    }

    notification.status = NotificationStatus::Pending;
    if (it == notifications_.end()) {
        NotificationRecord record;
        record.notification = std::move(notification);
        auto [record_it, inserted] =
            notifications_.emplace(key, std::move(record));
        IndexPending(record_it->second);
        return;
    }
    UnindexIfPresent(it->second);
    it->second.notification = std::move(notification);
    IndexPending(it->second);
}
```

Задача 4: отменённые задачи не отравляют hot path

SRE #4

Baseline делал lazy deletion: DELETE только добавлял id в cancelled_. После массовых отмен schedule_ оставался прежним, а due на каждом запросе снова проверял старые id.

Было

`cancel O(1), due O(N + C)`

Стало

`cancel O(log A), due O(K)`**БЫЛО:** marker set, мусор остаётся в schedule_

```
cancelled_.insert(key);
it->second.status = NotificationStatus::Cancelled;

if (cancelled_.count(id) != 0 || sent_.count(id) != 0) {
    continue;
}
```

СТАЛО: физическое удаление из due_index_

```
bool NotificationService::cancel(std::string_view id) {
    std::unique_lock<std::shared_mutex> lock(mu_);

    auto it = notifications_.find(std::string(id));
    if (it == notifications_.end()) {
        return false;
    }

    auto& record = it->second;
    if (record.notification.status ==
        NotificationStatus::Cancelled) {
        return true;
    }
    if (record.notification.status == NotificationStatus::Sent) {
        return false;
    }

    UnindexIfPresent(record);
    record.notification.status = NotificationStatus::Cancelled;
    return true;
}
```

Цена DELETE выросла до $O(\log A)$, но это осознанный обмен: редкая запись дешевле, чем бесконечный мусор в частом GET /due.

Задача 5: после terminal state id можно запланировать снова

Product #5

README: внешний id переиспользуется после sent или cancelled. При этом повторный POST для активной задачи должен оставаться идемпотентным.

Было

`sent_/cancelled_ скрывают id навсегда`

Стало

`status — единственный источник истины`

БЫЛО: terminal markers живут отдельно от Notification

```
sent_.insert(key);
it->second.status = NotificationStatus::Sent;

if (cancelled_.count(id) != 0 || sent_.count(id) != 0) {
    continue;
}
```

СТАЛО: идемпотентность только для pending

```
auto it = notifications_.find(notification.id);
if (it != notifications_.end() &&
    it->second.notification.status ==
        NotificationStatus::Pending) {
    return;
}

notification.status = NotificationStatus::Pending;

UnindexIfPresent(it->second);
it->second.notification = std::move(notification);
IndexPending(it->second);
```

Инвариант: sent/cancelled запись хранится для GET, но не участвует в due_index_. Новый POST после terminal заменяет её pending задачей.

Задача 6: один mutex создавал лишнюю очередь ожидания

SRE #6

В baseline все операции сериализованы через `std::mutex`. Дорогой due держал lock во время scan + sort и блокировал ingestion, cancel и markSent.

Было

`critical section:  $O(N + D \log D)$`



Стало

`read lock:  $O(K)$ , write lock:  $O(\log A)$`

БЫЛО: один lock для чтения и записи

```
bool NotificationService::markSent(std::string_view id) {
    std::lock_guard<std::mutex> lk(mu_);
    ...
}

std::vector<DueNotification> NotificationService::due(...) const {
    std::lock_guard<std::mutex> lk(mu_);
    ...
}
```

СТАЛО: shared_mutex + короткий hot path

```
std::optional<Notification> NotificationService::get(...) const {
    std::shared_lock lock(mu_);
    ...
}

std::vector<DueNotification> NotificationService::due(...) const {
    std::shared_lock lock(mu_);
    ...
}

bool NotificationService::markSent(...) {
    std::unique_lock lock(mu_);
    ...
}
```

Параллельные GET /due и GET /notifications не блокируют друг друга. Записи по-прежнему exclusive, потому что меняют map + index согласованно.

Задача 7: no-ready запрос должен быть дешёвым

Backend #7

Когда все уведомления запланированы на будущее, baseline всё равно обходил весь `schedule_`. Для очереди по времени достаточно посмотреть на самый ранний элемент.

Было

 $O(N)$ 

Стало

 $O(1)$ **БЫЛО:** future элементы не останавливают scan

```
for (const auto& id : schedule_) {
    auto it = notifications_.find(id);
    if (it == notifications_.end()) continue;
    const auto& n = it->second;
    if (n.send_at > now) {
        continue;
    }
    result.push_back(toDue(n));
}
```

СТАЛО: первый future элемент доказывает, что ready нет

```
for (auto it = due_index_.begin();
     it != due_index_.end() && result.size() < limit; ++it) {
    if (it->send_at > now) {
        break;
    }
    auto notification_it = notifications_.find(it->id);
    const auto& notification =
        notification_it->second.notification;
    result.push_back(toDue(notification));
}
```

Почему корректно: `due_index_` отсортирован по `send_at` первым ключом. Если `begin()->send_at > now`, то все остальные элементы тоже лежат в будущем.

Проверка после правок: тесты и запуск со скрина

13 / 13

GoogleTest после включения DISABLED_*

0 failed

./build -> ctest

DISABLED теперь проверяются и успешно проходят

ПОСЛЕ ПРАВOK

```
Start 8: NotificationServiceTest.ConcurrentAddSentAndDueIsSafe
8/13 Test #8: NotificationServiceTest.ConcurrentAddSentAndDueIsSafe .....
..... Passed 0.01 sec
Start 9: NotificationServiceTest.DueOrderingUsesPriorityCreatedAtAndId
9/13 Test #9: NotificationServiceTest.DueOrderingUsesPriorityCreatedAtAndId ..
..... Passed 0.00 sec
Start 10: NotificationServiceTest.DuplicateIdDoesNotCreateDuplicateDueEntr
ies
10/13 Test #10: NotificationServiceTest.DuplicateIdDoesNotCreateDuplicateDueEntr
ies ... Passed 0.00 sec
Start 11: NotificationServiceTest.CancelledIdCanBeScheduledAgain
11/13 Test #11: NotificationServiceTest.CancelledIdCanBeScheduledAgain .....
..... Passed 0.00 sec
Start 12: NotificationServiceTest.SentIdCanBeScheduledAgain
12/13 Test #12: NotificationServiceTest.SentIdCanBeScheduledAgain .....
..... Passed 0.00 sec
Start 13: NotificationServiceTest.DueOrderingUsesCreatedAtBeforeId
13/13 Test #13: NotificationServiceTest.DueOrderingUsesCreatedAtBeforeId .....
..... Passed 0.00 sec

100% tests passed, 0 tests failed out of 13
```

BASELINE

```
build - -zsn - 80x24

Start 11: NotificationServiceTest.CancelledIdCanBeScheduledAgain
11/13 Test #11: NotificationServiceTest.CancelledIdCanBeScheduledAgain .....
.....***Not Run (Disabled) 0.00 sec
Start 12: NotificationServiceTest.SentIdCanBeScheduledAgain
12/13 Test #12: NotificationServiceTest.SentIdCanBeScheduledAgain .....
.....***Not Run (Disabled) 0.00 sec
Start 13: NotificationServiceTest.DueOrderingUsesCreatedAtBeforeId
13/13 Test #13: NotificationServiceTest.DueOrderingUsesCreatedAtBeforeId .....
.....***Not Run (Disabled) 0.00 sec

100% tests passed, 0 tests failed out of 8

Total Test time (real) = 0.03 sec

The following tests did not run:
    9 - NotificationServiceTest.DueOrderingUsesPriorityCreatedAtAndId (Dis
abled)
    10 - NotificationServiceTest.DuplicateIdDoesNotCreateDuplicateDueEntrie
s (Disabled)
    11 - NotificationServiceTest.CancelledIdCanBeScheduledAgain (Disabled)
    12 - NotificationServiceTest.SentIdCanBeScheduledAgain (Disabled)
```

Метрики baseline до / после правок

BASELINE

```
itmo_cpp_notification_service — -zsh — 80x24

resp = self.send(prepare, **send_kwargs)
File "/Users/none/Downloads/itmo_cpp_notification_service/.venv/lib/python3.14/site-packages/requests/sessions.py", line 784, in send
r = adapter.send(request, **kwargs)
File "/Users/none/Downloads/itmo_cpp_notification_service/.venv/lib/python3.14/site-packages/requests/adapters.py", line 729, in send
raise ConnectionError(e, request=request)
requests.exceptions.ConnectionError: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /v1/notifications (Caused by NewConnectionError("HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 61] Connection refused"))

=====
Load test report (duration=70.8s, total=94427, errors=36896, rps=1334.6)
=====
due      : n= 38350 rps= 542.0 p50= 6.76ms p95= 117.04ms p99= 158.59ms
max=10108.99ms
add      : n= 29915 rps= 422.8 p50= 4.96ms p95= 115.71ms p99= 156.65ms
max=10103.62ms
sent     : n= 9077 rps= 128.3 p50= 5.32ms p95= 115.90ms p99= 158.82ms
max=10106.10ms
cancel  : n= 17085 rps= 241.5 p50= 4.94ms p95= 115.84ms p99= 158.39ms
max=10103.38ms
=====
(.venv) none@none itmo_cpp_notification_service %
```

ПОСЛЕ

```
itmo_cpp_notification_service-main 2 — -zsh — 80x24

resp = self.send(prepare, **send_kwargs)
File "/Users/none/Downloads/itmo_cpp_notification_service-main 3/.venv/lib/python3.14/site-packages/requests/sessions.py", line 784, in send
r = adapter.send(request, **kwargs)
File "/Users/none/Downloads/itmo_cpp_notification_service-main 3/.venv/lib/python3.14/site-packages/requests/adapters.py", line 729, in send
raise ConnectionError(e, request=request)
requests.exceptions.ConnectionError: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /v1/notifications (Caused by NewConnectionError("HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 61] Connection refused"))

=====
Load test report (duration=70.5s, total=541788, errors=36245, rps=7686.1)
=====
due      : n= 233558 rps= 3313.4 p50= 0.91ms p95= 17.50ms p99= 27.71ms
max=10025.76ms
add      : n= 181671 rps= 2577.3 p50= 0.91ms p95= 17.51ms p99= 27.40ms
max=10027.68ms
sent     : n= 22504 rps= 319.3 p50= 1.01ms p95= 23.24ms p99= 35.22ms
max=10027.90ms
cancel  : n= 104055 rps= 1476.2 p50= 0.90ms p95= 17.51ms p99= 27.41ms
max=10029.36ms
=====
(.venv) none@none itmo_cpp_notification_service-main 2 %
```

СРАВНЕНИЕ

Метрика	Было	Стало	Изменение
Total RPS	1334.6	7686.1	5.76× выше
due p50	6.76 ms	0.91 ms	7.43× быстрее
due p95	117.04 ms	17.50 ms	6.69× быстрее
due p99	158.59 ms	27.71 ms	5.72× быстрее
errors / total	39.1%	6.7%	5.84× ниже